



## **B4** Abstract data types (ADTs) (HL)

*Which ADTs are most appropriate for different situations?*

## SYLLABUS CONTENT

By the end of this chapter, you should be able to:

- ▶ B4.1.1 Explain the properties and purpose of abstract data types (ADTs) in programming
- ▶ B4.1.2 Evaluate linked lists
- ▶ B4.1.3 Construct and apply linked lists (singly, doubly and circular)
- ▶ B4.1.4 Explain the structures and properties of binary search trees (BST)
- ▶ B4.1.5 Construct and apply sets as an abstract data type (ADT)
- ▶ B4.1.6 Explain the core principles of abstract data types (ADTs)

## B4.1.1 Properties and purposes of abstract data types

◆ **Interface:** a contract that specifies a set of methods a class must implement, without defining how these methods are implemented, serving as a blueprint that promotes modularity, flexibility and abstraction in software development. This structure allows different classes to implement the same interface in diverse ways, while ensuring they provide the functionalities declared by the interface.

◆ **Modularity:** a design principle that involves dividing a system into distinct and manageable sections or modules, each with its own specific responsibilities, which can be developed, tested and maintained independently, but function cohesively when combined.

Abstract data types (ADTs) are fundamental constructs in programming that provide a theoretical framework for data manipulation through a clearly defined **interface**. ADTs embody the concept of abstraction by hiding the complexity of their operations from the user. Users interact with an ADT through a set of well-defined operations without needing to understand the underlying implementation details. This separation of interface from implementation allows programmers to focus on the “what” of the operations rather than the “how”, enhancing readability and maintainability.

Furthermore, encapsulation is integral to ADTs, safeguarding the data’s integrity by restricting direct access to the underlying data structure. The internal state of an ADT is accessed and modified solely through its methods, preventing unauthorized or harmful modifications to the data structure. This protective barrier ensures that the ADT operates reliably and as expected, regardless of the external use-case scenarios.

ADTs are defined by their behaviour rather than their physical implementation, allowing them to be applied universally across different programs and systems without modification. This property makes ADTs highly reusable and adaptable to various applications, promoting code reusability and reducing development time.

ADTs also exemplify the principle of **modularity**: the practice of decomposing complex systems into discrete, manageable components. This modularity facilitates debugging and testing, by isolating issues within discrete units without affecting the entire system. It also enhances the system’s scalability and understandability, making ADTs invaluable for building complex, robust applications.

### Common mistake

It is easy to get confused about the difference between interface and implementation. The interface is what operations are available, whereas the implementation is how these operations are carried out.

## TOK

### Areas of Knowledge (AOKs) and ADTs: Guiding questions

- How does abstraction in ADTs mirror abstraction in other disciplines?
- To what extent is modularity essential for organizing and simplifying complex systems?

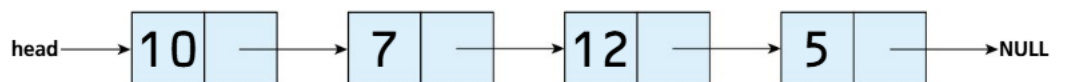
### ACTIVITY

Abstraction and modularity are key concepts in ADTs. In groups, investigate how abstraction and modularity are applied in their assigned AOK, such as natural sciences, arts and mathematics. Consider:

- How is abstraction used to simplify complex ideas?
- Are there risks or benefits to simplifying reality through abstraction?
- How does modularity help organize knowledge or creativity in these fields?

Present your findings and discuss as a class.

## B4.1.2 Linked lists



■ Linked list structure

A linked list consists of two main elements: nodes and pointers.

**Nodes** are the data element of the list. This could be a single piece of data, or it may be an object containing multiple data. There are two particularly important nodes: the *head* and the *tail*. The head is the first node in the linked list and the tail is the last. These help us navigate through the linked list.

**Pointers** are contained within the node. These point to the next node in the list (and sometimes the previous). A pointer may also be referred to as a *reference*, as it refers to the memory address of where the next node resides.

◆ **Node:** a basic unit of a data structure, e.g. a linked list or tree, which contains data and typically links to or references other nodes.

◆ **Pointer:** a variable that stores the memory address of another variable, typically used in programming to reference, or access, the location of data stored in memory.

### Top tip!



■ A trail of elephants (nodes), linked by their tails and trunks (pointers)

To visualize a linked list, you might imagine it as a chain of elephants, where each elephant represents a node. The trunk of one elephant extends to the tail of the next, similar to how pointers connect one node to another in a linked list.

## Advantages and disadvantages of linked lists

### Advantages

- **Dynamic data structures:** Due to their implementation, linked lists are dynamic data structures. This means that they can grow and shrink in size as we add and remove nodes. This is unlike an array, which is a static data structure.

◆ **Heap space:** a region of dynamically allocated memory managed by the operating system where programs store variables and data structures that require memory allocation during runtime, allowing for flexible memory usage that can grow and shrink as needed by the application.

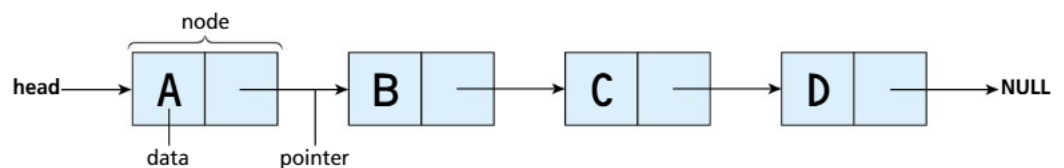
- **Memory utilization:** Linked lists can be more efficient with memory usage, as they do not create a reserved space in memory like an array does when it is declared.
- **Efficient insertion and deletion:** Linked lists generally outperform arrays when inserting or deleting data due to their dynamic nature.

### Disadvantages

- **Sequential access:** As linked lists cannot be accessed via an index like arrays, searching a linked list requires a linear search algorithm, where you start at the head node and continue traversing until you find what you are looking for. This means that using a binary search algorithm, for example, is not possible on a linked list. For large data structures, this restriction when searching can be very time consuming when compared to accessing an array.
- **Memory utilization:** Each node requires a reference to the next (and possibly the previous) node, as well as storing the data within it. This takes up system resources in the primary memory. As linked lists can grow, they potentially can go beyond the system resources that are available, causing a heap exhaustion as the system will run out of **heap space**.

## Types of linked lists

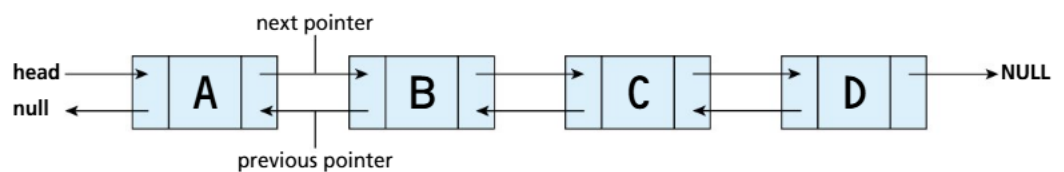
### Singly linked lists



#### ■ Singly linked list

This is the simplest of the three types. Each node contains a single pointer pointing to the next node in the list. The tail nodes contain a pointer that points to null (or none). This can be used to recognize the end of the list when traversing it.

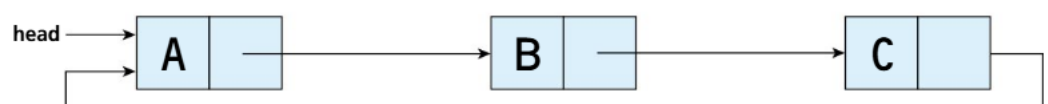
### Doubly linked lists



#### ■ Doubly linked list

These differ from singly linked lists because the nodes have two pointers – one reference to the next node in the list and one to the previous. This allows for easier traversing of the list, as you can move forward and backward with ease.

### Circular linked lists



#### ■ Circular linked list

These are similar to singly linked lists, but with one major difference: the tail node, instead of pointing to null, points back to the head node. This allows for circular, continuous traversal of the list. Doubly linked lists can also be implemented in this manner.

### REVIEW QUESTIONS

- 1 Sketch out a singly linked list containing the numbers 1 to 5.
- 2 Sketch out a doubly linked list containing the names of five of your classmates.
- 3 Sketch out a circular linked list containing the names of five of your idols.

### Common mistake

Do not forget to label the head and to point the tail node to null (or to the head node in a circular linked list).

### Top tip!

Remember to clearly show the node, as illustrated above. It should be represented as a box, divided into either two or three sections. The pointers should be arrows clearly indicating the node they are referring to.

## B4.1.3 Linked lists

### Linked list operations

These steps all refer to the operations based on a singly linked list.

#### Traversal / search

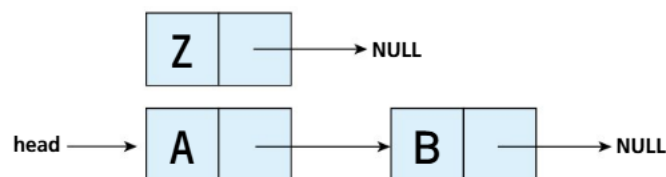
This is essentially a linear traversal / search, where you start at the beginning of the linked list and move along the list, following the pointers to the nodes.

- 1 Start with the head pointer to find the first node in the linked list.
- 2 From this node, follow the pointer within the node to the next node.
- 3 Repeat step 2 until you find the node you are searching for or until the pointer points to null / none, which means you have reached the end of the list.

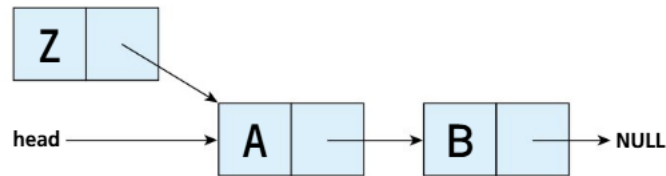
#### Insertion

There are three methods for inserting into a linked list. Which one you use depends on where you need to insert the node.

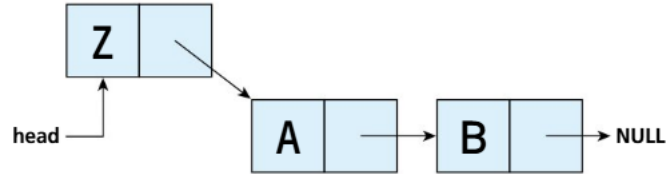
Inserting at the beginning:



- 1 Create a new node.

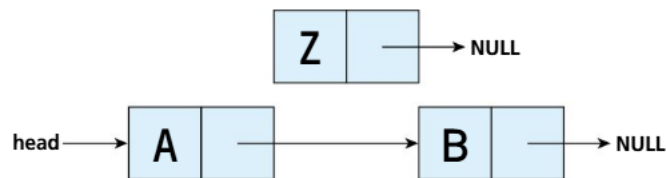


- 2 Point the new node to the current head node.

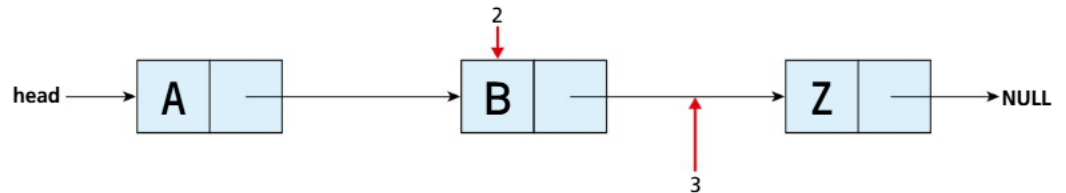


- 3 Update the head pointer of the linked list to the new node.

Inserting at the end:

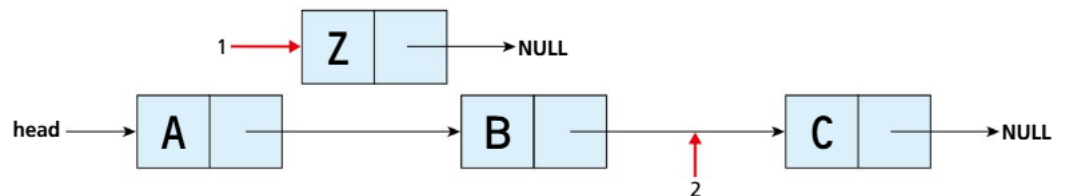


- 1 Create a new node.

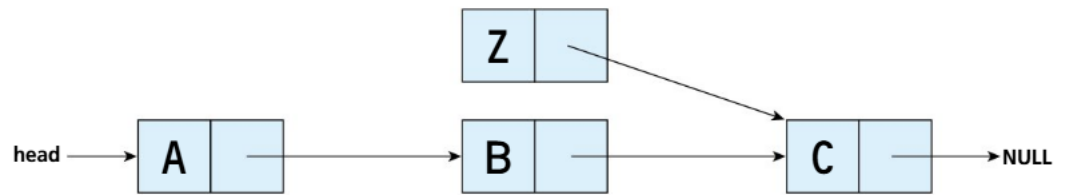


- 2 Find the last node by traversing the list until you find the tail node that points to null.
- 3 Point the current tail node to the new node (the new node pointer has not been set, so it will point to null).

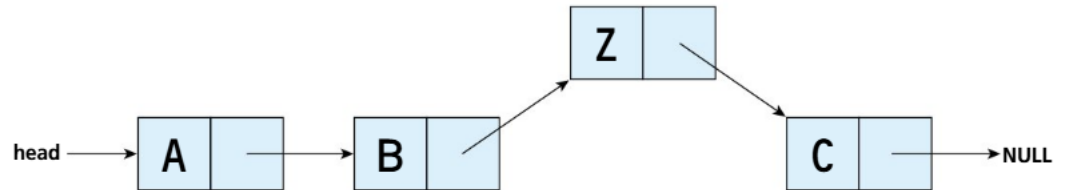
Inserting in the middle:



- 1 Create a new node.
- 2 Traverse the list to find the node after which you want to insert the new node.



- 3 Set the new node's pointer to point to the newly found node's pointer (at this point, both nodes will be pointing to the next node in the list).



- 4 Set the newly found node's pointer to the new node.

### Common mistake

Be careful not to complete action 4 before action 3 when inserting into the middle of the linked list. If you do this, you will lose any pointers to the second part of the list, and every node beyond the point of the insertion will be lost.

### Deletion

There are also three methods for deleting from a linked list. Which one you use depends on where you need to delete the node.

Deleting the first node:

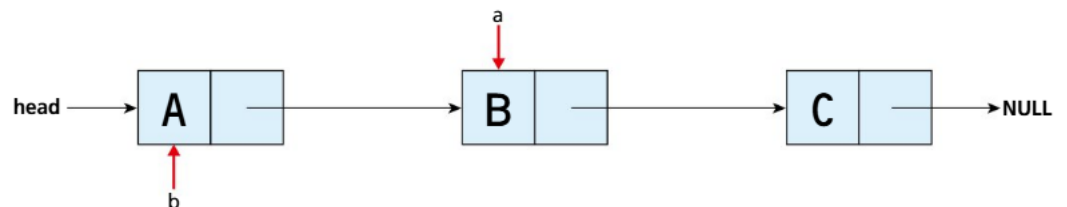


- 1 Check whether the list is empty. (If the head pointer is already null, the list is empty.)



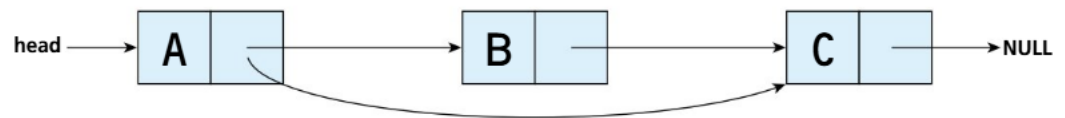
- 2 Set the head pointer to the new first node's pointer. (Now nothing is pointing to the original first node, which eliminates it from the list. Most high-level languages have garbage collection to realize this and clear it up without you having to delete it.)

Deleting a middle node:



- 1 a Traverse the list, starting from the head, to find the node you want to delete.

- b** While doing this, you will need to manually keep a pointer / reference to the previously visited node.



- 2** If you find the node you want to delete, use the previously visited node's pointer and adjust it to point to the found node's pointer. (Now nothing is pointing to the found node, which eliminates it from the list.)

Deleting the end node:



- 1 a** This process is similar to the deletion from a middle node. Traverse the list, starting from the head, until you find the tail node pointing to null.
- b** While doing this, you will need to manually keep a pointer / reference to the previously visited node.



- 2** Adjust the previously visited node's pointer to point to null (or none).

## REVIEW QUESTIONS

Draw a singly linked list with the following names: Aarav, Yuki, Sofia, Jamal and Elena.  
With the aid of diagrams:

- 1** Show the steps to insert Nia after Sofia.
- 2** Show the steps to delete Elena.
- 3** Show the steps to add Liam at the head.

## Construct linked lists

### Initial set-up and traversal

Each linked list has a class that is the starting point when creating a linked list. This often just includes a single instance variable: the pointer to the first node (the head).

We will also create a method that traverses the list to output the contents. Take note of how the loop works. We initially get access to the first node through the head pointer, and then we traverse through the nodes until we come to the tail node that points to null or none.

**Python**

```
class LinkedList:
    """ Constructor """
    def __init__(self):
        self.head = None # Initialize the head pointer
    """Function to print the linked list"""
    def print_list(self):
        current = self.head # Set to first node
        while current != None: # Loop until no further nodes
            print(current.data, end=" -> ") # Output node data
            current = current.next # Move pointer to next node
        print("None")
```

**Java**

```
public class LinkedList {
    ListNode head; // Head of the list
    // Method to print the LinkedList
    public void printList() {
        ListNode n = head; // Set to first node
        while (n != null) { // Loop until no further nodes
            System.out.print(n.data + " -> "); // Output node
            // data
            n = n.next; // Move pointer to next node
        }
        System.out.println("NULL");
    }
}
```

We then create a node class that contains data and at least one pointer. If we were creating a doubly linked list, we would have a second variable (`self.previous` for Python, or `ListNode previous` in Java).

**Python**

```
class ListNode:
    """ Constructor (1 parameter data with a default value of 0) """
    def __init__(self, data=0):
        self.data = data
        self.next = None
```

**Java**

```

class ListNode {
    int data; // This example will create a node with data of type int, but you
    // may use any type here depending on your needs
    ListNode next;
    // Constructor to create a new node
    ListNode(int d) {
        data = d;
        next = null; // Set to null
    }
}

```

**Insertion**

The insertion method should be created inside the LinkedList class. This should have a parameter to receive the data passed to it. From there, it should create a new node with that data and insert it into the correct position in the list. Remember that there are three different insertion methods: at the beginning, in the middle or at the end.

The `insert_after_value` method assumes we want to insert after a found value. This could be implemented in different ways. It could be inserted in a certain position or after a node that has already been identified and passed as a parameter.

**Python**

```

def insert_at_beginning(self, data):
    new_node = ListNode(data) # Create the new node with the data given
    new_node.next = self.head # Set new node pointer to current head node
    self.head = new_node # Set head pointer to new node
    """
    Inserts a new node with 'data' after the first node found with 'target_
    value'.
    If 'target_value' is not found, does not insert the new node.
    """

def insert_after_value(self, target_value, data):
    current = self.head
    while current is not None:
        if current.data == target_value: # If we find the node to insert after
            new_node = ListNode(data) # Create a new node with provided data
            new_node.next = current.next # Set new node pointer to the same
            # pointer as the found node
            current.next = new_node # Set the found node pointer to the new
            # node
            return # We can now end the method early
        current = current.next # Otherwise move to the next node
    print(f"Node with data {target_value} not found.") # If loop ends we did
    # not find our target

```

```

def insert_at_end(self, data):
    new_node = ListNode(data) # Create new node
    if self.head is None: # Check if list is empty
        self.head = new_node # Insert new node as head of list
    else:
        current = self.head # Get a link to the first node in the list
        while current.next != None: # Keep moving down the list
            current = current.next
        # When the above loop ends, the next pointer must be None, indicating we
        # have reached the last node
        current.next = new_node # Set pointer on current last node to the new node

```

## Java

```

public void insertAtBeginning(int data) {
    ListNode newNode = new ListNode(data); // Create a new node with the data given
    newNode.next = head; // Set new node pointer to current head node
    head = newNode; // Set head pointer to new node
}

public void insertAfter(int targetValue, int data) {
    ListNode current = head;
    // Traverse the list to find the target value
    while (current != null && current.data != targetValue) {
        current = current.next;
    }
    // At this point current will either be null (not found) or our target node
    // If the target node is found, insert the new node after it
    if (current != null) {
        ListNode newNode = new ListNode(data); // Create new node with data
        // provided
        newNode.next = current.next; // Set new node pointer to the same pointer
        // as the found node
        current.next = newNode; // Set the found node pointer to the new node
    } else {
        // If the target value is not found in the list
        System.out.println("Node with value " + targetValue + " not found.");
    }
}

public void insertAtEnd(int data) {
    ListNode newNode = new ListNode(data); // Create a new node
    if (head == null) { // Check if list is empty
        head = newNode; // Insert new node as head of list
    } else {
        ListNode last = head; // Get link to the first node in the list
        while (last.next != null) { // Keep moving down the list
            last = last.next;
        }
    }
}

```

```

        // When the above loop ends, the next pointer must be null, indicating
        // we have reached the last node
        last.next = newNode; // Set pointer on current last node to the new node
    }
}

```

### Deletion

Like the insertion methods, the deletion method will also reside in the `LinkedList` class. We assume here that all items in our list are unique. If this were not the case, these methods would delete the first occurrence. Our method has one parameter: the data we are looking for.

Our delete method must be prepared for three possible outcomes:

- 1 We are deleting the head node
- 2 We are deleting a middle node or the tail node
- 3 The node may not be found.

### Python

```

def delete_node(self, data):
    current = self.head # Get the head node
    prev = None
    # Case 1: If the node to be deleted is the head of the list
    if current != None and current.data == data:
        self.head = current.next # Change head to the current head's pointer
        current = None # Free the old head
        return
    # Search for the node to be deleted, remembering to keep track of the
    # previous node
    while current != None and current.data != data:
        prev = current
        current = current.next
    # Case 3: If the node is not found
    if current == None:
        print(f"Node with data {data} not found.")
        return
    # Case 2: Unlink the node from the list
    prev.next = current.next
    current = None

```

**Java**

```

public void deleteNode(int data) {
    ListNode current = head
    prev = null;
    // Case 1: If the head node is to be deleted
    if (current != null && current.data == data) {
        head = current.next; // Changed head
        return;
    }
    // Case 2: If the node to be deleted is somewhere other than at the head
    while (current != null && current.data != key) {
        prev = current;
        current = current.next;
    }
    // Case 3: If the key is not present in the list
    if (current == null) {
        System.out.println("Node with value " + key + " not found.");
        return;
    }
    // Unlink the node from the linked list
    prev.next = current.next;
}

```

**Search**

The search method also resides in the LinkedList class. This is a relatively straightforward method that combines some of the techniques we have already used. The method needs a single parameter to look for and, in these examples, will return true or false, depending on whether the item was found. This could be modified to return the position in the list of the node itself, if needed.

**Python**

```

def search(self, key):
    current = self.head # Get the head node
    while current != None: # Keep looping until we have run out of nodes
        if current.data == key: # If we find the node we are looking for
            return True
        current = current.next # Move to the next node
    return False # Value not found in the list

```

**Java**

```

public boolean search(int key) {
    ListNode current = head; // Get the head node
    while (current != null) { // Keep looping until we have run out of nodes
        if (current.data == key) { // If we find the node we are looking for
            return true;
        }
        current = current.next; // Move to the next node
    }
    return false; // Value not found in the list
}

```

**REVIEW QUESTION**

Create the LinkedList class and ListNode class. Using a test table, plan out the actions you will take to ensure that these list methods are working correctly. When you have done that, carry out the tests.

## B4.1.4 Structures and properties of binary search trees

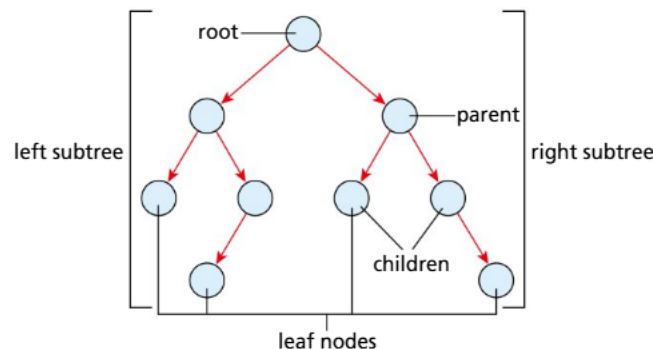
### ■ How binary search trees are used for data organization

A tree structure in Computer Science is used to hold data in order and is usually drawn upside down, with the root at the top and the leaves at the bottom.

◆ **Root:** the topmost node from which all other nodes descend, serving as the starting point for any traversal or operation within a binary search tree.

◆ **Parent:** a node that has one or more nodes directly beneath it, connected by edges, and it directly controls these subsequent child nodes.

◆ **Child:** any node that has a direct link from a parent node positioned above it, potentially having further child nodes of its own.



### ■ Tree structure and its parts

In this section, we will be focusing on a tree that follows some strict rules, which allow us to optimize search, insertion and deletion operations.

However, before we discuss the rules of binary search trees (BST), it is important to understand the terminology. The node at the top of the tree is known as the **root** node. If a node has a node attached below it, it is known as a **parent** node, with the one below being the **child** node. Each parent node can have one or two child nodes. The child node that is less than the parent goes on the left; the child node that is greater than the parent, on the right.

◆ **Subtree:** any node, along with its descendants, functioning as a standalone binary search tree, with its node acting as the root.

◆ **Leaf:** a node that does not have any children, representing the endpoints of a binary search tree's branches.

The sub-section to the right of a node is known as the right **subtree** and to the left, the left subtree. All nodes at the bottom of the tree, without children, are known as **leaves**.

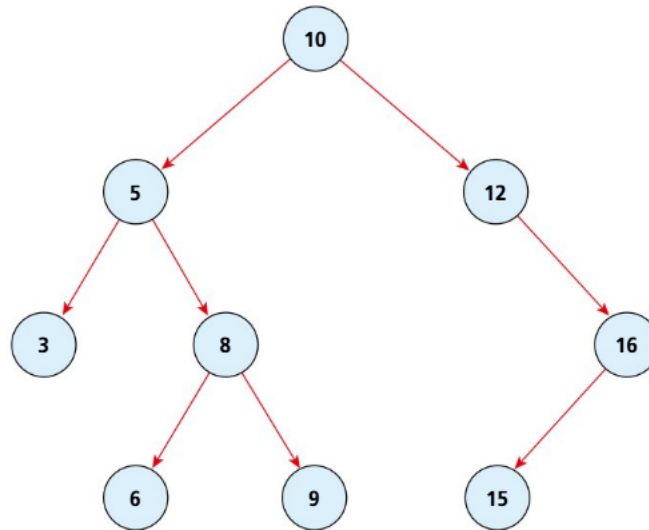
We can assume that all items within the BST are unique (there are no duplicates).

## Common mistake

Make sure you move in the correct direction when navigating through the tree:

- Move **left** if the item is smaller than the node.
- Move **right** if the item is larger than the node.

## Tree structure and node insertion



Here, you can see these two rules in place. In this example, 10 was the first node entered into the BST and set as the root. Each new node works its way through the tree until it finds a space where it can join. We cannot be completely sure what was entered next, but let us assume that it was node 5:

- Node 5: This would be compared to node 10; as it is smaller than node 10, it would join to the left of that node. This is now a child of node 10, which is the parent node of node 5. At this point, node 5 is also a leaf, but this will change as more nodes are added later.
- Node 12: This would be compared with 10 and, as it is greater, it would join to the right. This is now a child of node 10, which is the parent node of nodes 5 and 12.
- Node 3: This would be compared to node 10; as it is smaller, it would move to the left. It would then be compared with node 5 and, as it is smaller, it would be moved to the left again, where it would join the tree. This is now a child of node 5, and node 5 is the parent of node 3.
- Node 8: This is smaller than node 10, so it moves to the left; it is greater than node 5, so it moves to the right. This is now a child of node 5, and node 5 is the parent of nodes 3 and 8.

## Common mistake

Leaves are not always at the bottom of the tree. A leaf is any node in the tree that does not have any children.

## REVIEW QUESTIONS

- 1 Write the statements for how nodes 6, 9, 16 and 15 join the tree.
- 2 Sketch the resulting binary tree when the following items are entered:  
10, 15, 3, 12, 7, 1, 22, 18, 5
- 3 Which nodes are:
  - a parents?
  - b children?
  - c leaves?
  - d right subtree?
  - e left subtree?
  - f root?

## Node search

To find a node in a BST, we follow these steps, looking for the key we are searching for:

- 1 **Start at the root node:** Begin your search from the root of the BST.
- 2 **Check for null / none:** If the current node is null (or none in Python), the search concludes without finding the key. The key is not present in the BST.
- 3 **Compare the node value with the key:**
  - a If the current node's value matches the key, the search is successful. The key is found in the BST.
  - b If the key is smaller than the current node's value, proceed to the left child of the current node.
  - c If the key is larger than the current node's value, move to the right child of the current node.
- 4 **Repeat the process:** Continue the process from step 2 with the new current node.

## Node traversal

Depending on the requirements, there are different ways to traverse the BST and return the data within. These are:

- in-order traversal
- pre-order traversal
- post-order traversal.

To perform these operations, it is important to remember the order of actions as you move through the BST. The easiest way to do this is to remember that you *always* move left before right. After that, you just need to remember when you output the node data.

## Top tip!

The traversal name gives you a clue to where the node check is:

- Pre-order: check before left and right (NLR)
- Post-order: check after left and right (LRN)
- In-order: goes in the middle (LNR).

Let us look again at the BST illustrated above.

### In-order traversal (left, node, right)

We start at the root (10) and need to perform all three operations on this node. We start with the first, **left**. This takes us to node 5 with the same situation, **left** and we move to node 3. Here, we move **left** (no further node), **node** so we output 3, and then move **right** (no further node). We have now completed all three actions on this node. So, we move back to node 5. Here we have already gone left, so now we output **node**, which would be 5, and then we move **right**. We continue with these operations until all three have been completed on every node.

The final output would be:

3, 5, 6, 8, 9, 10, 12, 15, 16

You will notice the numbers are output in order. This is the main purpose of an in-order traversal – to output the nodes, sorted, in ascending order.

### Pre-order traversal (node, left, right)

We carry out the same operations here, but in a different order. We start again at the root (10), but this time we output **node** value first. We then move **left** to node 5, where we output the **node** value there. We move **left** again and output **node** 3. We move **left** and **right** on node 3, completing the operations, and then move back to node 5, where we move **right** to node 8. We then continue in this manner until all three operations have been completed on every node.

The final output would be:

10, 5, 3, 8, 6, 9, 12, 16, 15

This method is useful for creating a copy of the tree or for exploring paths, as it visits the parent prior to the children.

### Post-order traversal (left, right, node)

Once you have understood the first two methods, this third should not cause any problems. It follows the same idea again, but this time we visit the node last. We start at the root (10), and we go **left** to node 5. We then go **left** again to node 3, where we complete all three operations, **left**, **right** and then finally **node**, where 3 is output. We then go back to node 5 and move **right** to node 8, where we go **left** to node 6 and complete all three actions, with 6 being output. We then move back to node 8 and move **right** to node 9. We then continue in this manner until all three operations have been completed on every node.

The final output would be:

3, 6, 9, 8, 5, 15, 16, 12, 10

This method is often used for deleting nodes within the tree as it visits children before their respective parent nodes.

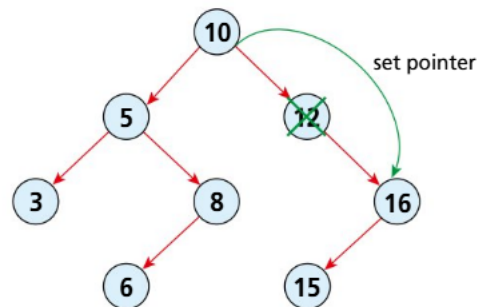
## REVIEW QUESTIONS

- 1 On the BST you drew for the previous review questions, carry out the following and show the output:
  - a In-order traversal
  - b Pre-order traversal
  - c Post-order traversal
- 2 What would happen if non-numerical data were input into a BST? Sketch the following BST when the data is input in this order:  
Ava, Alex, Bella, Catherine, Carlos, Brian, Aaron, Chloe

## ■ Node deletion

To delete a node in a BST, we take the following steps, looking for the key to delete and then carrying out the action based on the situation we find ourselves in:

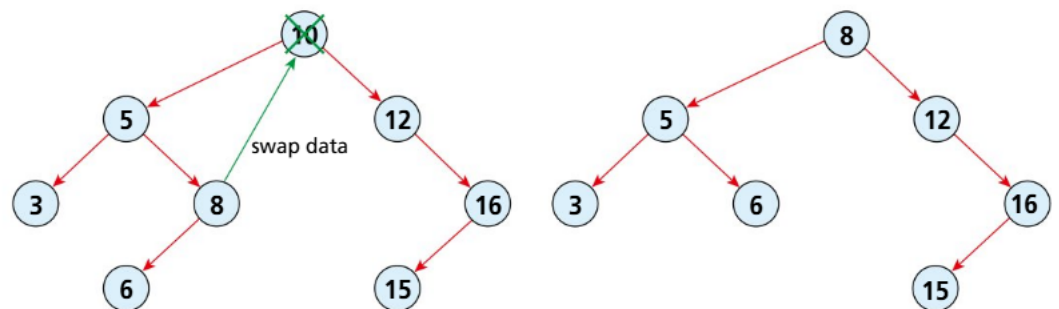
- 1 **Start at the root node:** Begin the deletion process from the root of the BST.
- 2 **Search for the key:** Follow the algorithm from the node search.
- 3 **Once the node is found, determine its type:**
  - a **Leaf node (no children):** if the node has no left or right child
  - b **One child:** if the node has exactly one child (either left or right)
  - c **Two children:** if the node has both a left and a right child.
- 4 **Delete the node based on its type:**
  - a **Leaf node:** Simply remove the node from the tree by setting its parent's appropriate (left or right) child pointer to null.
  - b **One child:** Bypass the node by linking its parent directly to its child. If the node is a left child, update the parent's left pointer; if a right child, update the parent's right pointer.



■ Deletion of node with one child

### c Two children:

- i Find the in-order successor (smallest value in the right subtree) or the in-order predecessor (largest value in the left subtree).
- ii Replace the value of the node to be deleted with the in-order successor's (or predecessor's) value.
- iii Delete the in-order successor (or predecessor) by repeating step 4, which now becomes a case of deleting a node with at most one child.



■ Deletion of node with two children

### 5 Repeat the process as needed:

If you had to delete the in-order successor or predecessor (step 4c), repeat the deletion process for that node.

## ACTIVITY

### Thinking skills: Problem-solving and analysis

Coding competitions are an effective way to improve your programming and problem-solving skills. Here is a question from CodingQuest where a BST can be used to solve it:  
<https://codingquest.io/problem/26>

## B4.1.5 Sets as an abstract data type

◆ **Unordered set:** a collection of unique elements where the elements do not have a specific order or sequence and their arrangement can vary each time they are accessed.

◆ **Mutable:** a set whose state or content can be changed after it has been created, allowing for modifications, e.g. adding, removing or altering elements within the object.

Like lists and arrays, sets can be used to store multiple values in a single variable. Sets are **unordered**, meaning that they cannot be accessed via index or key, but they are **mutable**, allowing the addition and removal of items. However, the individual items cannot be updated or change values. Sets contain only unique elements.

They are a powerful data structure when unique elements and efficient operations are crucial. They are extremely useful when you want to ensure a collection consists of only unique elements, such as when working with usernames where duplicates are not allowed. They are also extremely fast when needing to check whether an item is already part of a collection. When needing to perform mathematical set operations, such as union, intersection and difference, which are commonly used with database queries, search algorithms and data analysis, sets perform very well.

Key characteristics of sets are that:

- they are unordered
- they are mutable
- they contain unchangeable values
- they contain only unique elements.

### ■ Creating sets

There are two ways of creating a set in Python:

#### Python

```
# Using curly brackets
my_set = {1, 2, 3, 4, 5}
# Using the set() constructor
my_set2 = set([1, 2, 2, 3, 4, 5]) # Duplicate entries will
# automatically be removed
```

In Java, there are a number of classes that implement the `Set` interface. For our example purposes, we will use the `HashSet`. The other possibilities are `LinkedHashSet` and `TreeSet`, which will not be covered here. Using `HashSet` requires the import of two libraries:

#### Java

```
import java.util.HashSet;
import java.util.Set;
// Creating a HashSet of type Integer
Set<Integer> mySet = new HashSet<>();
```

## ■ Set methods

To add and remove elements:

### Python

```
my_set = {1, 2, 3, 4, 5}
my_set.add(6) # Add 6 to the set
my_set.remove(1) # Remove 1 from the set
```

### Java

```
Set<Integer> mySet = new HashSet<>();
my_set.add(6); // Add 6 to the set
my_set.remove(1) // Remove 1 from the set
```

To check whether elements are present in the set:

### Python

```
if 2 in my_set:
    print("2 is in the set")
else:
    print("2 is not in the set")
```

### Java

```
if (my_set.contains(2)) {
    System.out.println("2 is in the set.");
} else {
    System.out.println("2 is not in the set.");
}
```

◆ **Set union:** the union of two sets is a new set containing all the elements that are in either of the original sets, effectively combining them without any duplicate elements.

◆ **Set intersection:** the intersection of two sets is a new set containing only the elements that are present in both of the original sets, identifying their common elements.

◆ **Set difference:** the difference between two sets is a new set containing elements that are in the first set but not in the second set, effectively subtracting the elements of the second set from the first.

## ■ Set operations

There are three main operations that are used to manipulate and compare sets. These are **union**, **intersection** and **difference**. We will show their differences using the set created below.

### Python

```
# Define two sets
A = {1, 2, 3, 4}
B = {3, 4, 5, 6}
```

### Java

```
// Define two sets
Set<Integer> A = new HashSet<>();
A.add(1);
A.add(2);
A.add(3);
A.add(4);
Set<Integer> B = new HashSet<>();
B.add(3);
B.add(4);
B.add(5);
B.add(6);
```

## Union

A union joins two sets to create one that contains all the elements from both, without any duplicates.

### Python

```
# Union
union_set = A | B # or A.union(B)
print("Union:", union_set)
# Output
# Union: {1, 2, 3, 4, 5, 6}
```

### Java

```
// Union
Set<Integer> unionSet = new HashSet<>(A); // Creates a new
// set containing all elements from A
unionSet.addAll(B); // Performs a union join with B
System.out.println("Union: " + unionSet); // Output set contents
// Output
// Union: [1, 2, 3, 4, 5, 6]
```

## Intersection

An intersection of two sets creates a new set that contains only the elements that are present in both.

### Python

```
# Intersection
intersection_set = A & B # or A.intersection(B)
print("Intersection:", intersection_set)
# Output
# Intersection: [3, 4]
```

### Java

```
// Intersection
Set<Integer> intersectionSet = new HashSet<>(A);
intersectionSet.retainAll(B);
System.out.println("Intersection: " + intersectionSet);
// Output
// Intersection: [3, 4]
```

## Difference

The difference between two sets is a set containing elements that are in the first set but not in the second.

### Python

```
# Difference
difference_set = A - B # or A.difference(B)
print("Difference:", difference_set)
# Output
# Difference: [1, 2]
```

### Java

```
// Difference
Set<Integer> differenceSet = new HashSet<>(A);
differenceSet.removeAll(B);
System.out.println("Difference: " + differenceSet);
// Output
// Difference: [1, 2]
```

## REVIEW QUESTIONS

Set A: {4, 8, 15, 16, 23}

Set B: {42, 8, 16, 60, 7}

- 1 Show the output if a union operation is performed.
- 2 Show the output if an intersection operation is performed.
- 3 Show the output if a difference operation is performed.

## ACTIVITY

### Social Skills: Collaborative group work

#### Social-media friend recommendation

With a partner, working in your chosen language, create two sets consisting of the following names:

A: Carlos Gomez, Yuna Kim, Dmitri Ivanov

B: Yuna Kim, Dmitri Ivanov, Leila Al-Farsi, Sean O'Brien

A and B represent two users on a social-media network. A and B are friends with each other.

- 1 Using set operations, identify the users' common friends.
- 2 Using set operations, identify friends of B that are not friends with A, so the platform can recommend new connections.



◆ **Set subset:** a set where all elements of this set are also elements of another set, indicating that the first set is entirely contained within the second set.

## ■ Check whether subset or superset

A **subset** or **superset** describes the relationship between two sets.

Set A is considered a **subset** of B if all of A's elements are present in B – which would be considered the **superset**. All elements of the subset can be found in the superset.

To check whether set A is a subset of set B, or whether B is a superset of A:

### Python

```
# Define two new sets
A = {1, 4, 7}
B = {1, 2, 3, 4, 5, 6, 7}
# Check if A is a subset of B
# Both of these methods are acceptable
print(A.issubset(B)) # True
print(A <= B) # True
# Check if B is a superset of A
# Both of these methods are acceptable
print(B.issuperset(A)) # True
print(B >= A) # True
```

### Java

```
// Define two new sets
Set<Integer> A = new HashSet<>();
A.add(1);
A.add(4);
A.add(7);
Set<Integer> B = new HashSet<>();
B.add(1);
B.add(2);
B.add(3);
B.add(4);
B.add(5);
B.add(6);
B.add(7);
// Check if A is a subset of B
System.out.println(B.containsAll(A)); // True
// Check if B is a superset of A by checking if A is contained within B, but B
// is not contained within A
System.out.println(B.containsAll(A) && !A.containsAll(B)); // False
```

## B4.1.6 Core principles of abstract data types

### ■ Hash tables

Hash tables are a particularly important data structure in Computer Science. They offer rapid retrieval and insertion capabilities into an array-like structure. However, rather than an abstract integer being used by the programmer for the index, a key can be provided instead.

◆ **Hashing algorithm:**

a function that converts input data of any size into a fixed-size string of characters, which typically represents the data in a compressed and seemingly random format and is used primarily for indexing and retrieving items in databases more efficiently.

◆ **O(1) time**

**complexity:** describes an algorithm that takes the same amount of time to execute regardless of the size of the input data set.

◆ **ASCII (American Standard Code for Information Interchange):**

a character-encoding standard used to represent text in computers and other devices, defining a numerical value for each symbol and character commonly used in the English language.

This key is then processed through a **hashing algorithm** to find which array index to store the data in. This allows the data to be stored in an array-like structure that provides an **O(1) average-time complexity** for search, insert and delete operations under ideal conditions.

For example, “name” could be used as the key by the developer. A hashing algorithm then processes this with the aim of producing an integer so that where the data this key is linked to can be stored in the array. One example method where this could be achieved is by adding up the **ASCII** values of all the letters and then using modulus and the size of the table to determine the index.

```
# Hash table size
table_size = 10
# ASCII values for letters
'n' = 110
'a' = 97
'm' = 109
'e' = 101
110 + 97 + 109 + 101 = 417
417 % table_size = 7
```

This data would then be stored at index 7.

### Hash tables: Creating, inserting and retrieving data

This is how we can create a hash table, as well as insert, delete and retrieve items:

#### Python

```
# Creating a hash table as a Python dictionary
hash_table = {}
# Inserting two items
hash_table["key1"] = "value1"
hash_table["key2"] = "value2"
# Retrieving and printing one item
value = hash_table["key1"]
print("The value for 'key1' is:", value)
# Deleting an item
del hash_table["key2"]
```

#### Java

```
import java.util.HashMap;
public class Main {
    public static void main(String[] args) {
        // Creating a hash table using HashMap
        // The first data type is for the key, the second is for the data
        HashMap<String, String> hashTable = new HashMap<>();
        // Inserting two items
        hashTable.put("key1", "value1");
        hashTable.put("key2", "value2");
```

```

// Retrieving and printing one item
String value = hashTable.get("key1");
System.out.println("The value for 'key1' is: " + value);
// Deleting an item by key
String removedValue = hashTable.remove("key2"); // Returns the value
// associated with the key
    }
}

```

### Collision factors

As you may have already guessed, the hashing algorithm demonstrated above is not foolproof. There could be other example keys that also generate the number 7 for the index – when this happens it is called a “collision”. Ideally, our hashing algorithm should generate a unique index for each key and, while there are some more complex algorithms than the one we looked at that are better at doing this, there is no complete solution available. We cannot store two data items in an array at the same index, so how do we get around this problem? There are two main methods used: “chaining” and “open addressing”. Before we look at these methods, it is important to understand the impact of the number of items we are trying to store in the hash table. This is called the “load factor”.

### Load factors

The load factor is a measure that indicates how full the hash table is. The load factor is defined as the ratio of the number of elements currently stored in the table to the total number of slots available. The formula for load factor is:

$$\text{Load factor} = \frac{\text{number of entries in the table}}{\text{total number of slots}}$$

The load factor can heavily affect the performance of the hash table. A hash table with a high load factor will have more collisions, which will impact performance when performing operations on the table.

### Rehashing

When the load factor exceeds a certain threshold, **rehashing** is necessary to maintain efficient performance. Rehashing involves creating a new, larger array and redistributing the existing elements using a new hash function or the same hash function applied to the new array size. This process reduces the load factor and minimizes collisions, ensuring that the hash table operations remain efficient.

#### Steps involved in rehashing:

- 1 Monitor the load factor of the hash table and, when this exceeds 0.7 (70 per cent), trigger rehashing.
- 2 Prepare a new array that is at least double the size of the current array. Ideally, the new size should be a prime number to help reduce collisions.
- 3 For each element already in the hash table, all new hash values need to be computed based on the new array size.
- 4 Insert the elements into the new, larger array, ensuring that any collisions are handled appropriately.

◆ **Rehashing:** a process in hash tables where the data is redistributed into a new, larger array to reduce the load factor and minimize collisions, maintaining efficient performance.

### Common mistake

Do not overlook the impact of a high load factor on hash-table performance. Keeping the load factor low is incredibly important for efficiency.

◆ **Hash table chaining:**

a collision-resolution technique in hash tables where each bucket or index in the array can store multiple elements in the form of a linked list, allowing more than one entry to be stored at the same index.

◆ **High load factors (hash tables):**

a condition where a sizeable portion of the hash table's slots are filled, leading to increased collisions and potentially degraded performance, due to more frequent need for collision resolution mechanisms.

◆ **Open addressing:**

a collision resolution method in hash tables where, instead of using structures like linked lists to store multiple items at the same index, any colliding item is placed into the next available open slot in the hash table itself, according to a probing sequence.

## Chaining

**Chaining** utilizes linked lists (or a similar structure) to be able to store more than one item of data at a single index. When a collision occurs, and two keys hash to the same index, the new key-value pair is added to the end of the list at that index.

Let us consider a simple hash function as “key mod 6” and a sequence of keys as 35, 800, 82, 92, 122 and 94.

■ **Empty table**

|   |  |
|---|--|
| 0 |  |
| 1 |  |
| 2 |  |
| 3 |  |
| 4 |  |
| 5 |  |

■ **Insert 35**

|   |    |
|---|----|
| 0 |    |
| 1 |    |
| 2 |    |
| 3 |    |
| 4 |    |
| 5 | 35 |

■ **Insert 800 and 82**

|   |     |
|---|-----|
| 0 |     |
| 1 |     |
| 2 | 800 |
| 3 |     |
| 4 | 82  |
| 5 | 35  |

■ **Insert 122: Collision, so it is added to a chain**

|   |     |    |     |
|---|-----|----|-----|
| 0 |     |    |     |
| 1 |     |    |     |
| 2 | 800 | 92 | 122 |
| 3 |     |    |     |
| 4 | 82  |    |     |
| 5 | 35  |    |     |

■ **Insert 94: Collision, so it is added to a chain**

|   |     |    |     |
|---|-----|----|-----|
| 0 |     |    |     |
| 1 |     |    |     |
| 2 | 800 | 92 | 122 |
| 3 |     |    |     |
| 4 | 82  | 94 |     |
| 5 | 35  |    |     |

### Advantages:

- **Simplicity:** It is relatively easy to implement.
- **Handles high load factors well:** If the number of items you are inserting is greater than the amount of spaces in the table, it will still operate efficiently. However, lookup times may become slower when having to access a linked list.
- **Good for unknown data sizes** (connected to the point above): If you do not know in advance how many items you will be inserting into the table, chaining will have an advantage as you will not need to rehash the table to resize it.

### Disadvantages:

- **Memory overhead:** As it is utilizing a linked list, the more items there are in the chain, the more memory it will require.
- **Complexity for deletion:** If the deletion is within a linked list, performance will degrade due to the need to adjust pointers.
- **Variable performance:** When accessing an index with a linked list, this will perform worse than one without, which can impact search and delete actions.

## Open addressing

**Open addressing** only stores data within the hash table itself. When a collision occurs, it will find another empty slot in the hash table, according to a predefined sequence, and store the data there.

Several methods can be used to achieve this:

- **Linear probing** sequentially checks the next spot until a space is found. However, this can lead to clustering, where a group of adjacent slots get filled, increasing the search time for these elements in particular.

- **Quadratic probing** searches in a more spaced-out manner for an available slot using the original hash value and a quadratic function. If the next space is also full, it increments the value of the quadratic function and searches again until it finds an available slot.
- **Double hashing** uses a second hash function to determine the probe step. The first hash function is performed first, and then a second one to give an offset from the original index. This offers better distribution and minimizes clustering compared to linear and quadratic probing, but requires more computational overhead.

#### Advantages:

- **Space efficient:** Stores all elements directly within the hash table array, eliminating the need for extra data structures, such as linked lists.
- **More memory efficient:** No pointers are required.
- **Simpler to serialize:** As it is a simpler data structure with contiguous memory allocation, converting the structure to a format to be stored or transmitted is simpler than a hash table using chaining.

#### Disadvantages:

- **Increased computational overhead:** If there are high load factors, performance may decrease due to the probing methods required.
- **Clustering:** This can be an issue especially when using linear probing, where consecutive slots are filled, which increases the average time for insertions, deletions and searches that do not find the element they are searching for.
- **Complex deletion:** Deleting an element is a complex process as you may break probe sequences.

### PROGRAMMING EXERCISE 1

Using a hash table, create a simple voting system. The system should have the ability to add and remove candidates. Use the name input by the user as the key in the hash table. Once the candidates have been input, it should allow voters to cast votes for the candidates in the system and allow the current totals to be viewed. There should be an option to end the election when the voting is over, and the winner should be output to the user.

### PROGRAMMING EXERCISE 2

Working in a team, conduct a small experiment by implementing two different ADTs to solve the same problem and compare the performance. You could consider execution time and memory usage, for example.

Collaborate on this project by assigning roles, managing your time and co-operating to achieve your goal. You could create a shared code repository to help you work together.

When you have finished, prepare a presentation summarizing your project. Include the problem definition, implementation details, test results, analysis and conclusions.

An example project:

#### Spell checker

Implement a spell checker using a hash table and a binary search tree. Compare the performance for the following operations:

- Inserting words into the dictionary.
- Checking whether a word is in the dictionary.
- Suggesting corrections for a misspelled word (finding the nearest match).

## ● Linking questions

- 1 What role do stacks and queues play in handling CPU interrupts and polling? (A1)
- 2 Can abstract data types be used to manage data? (A2)
- 3 How can abstract data types be used to optimize the file-processing operations like read and write? (B2)
- 4 Can a binary search tree play a role in the quicksort algorithm? (B1)

## EXAM PRACTICE QUESTIONS

- 1 Construct a diagram to represent a double-linked list that holds the following sequence of names:  
Kaja, Aiko, Carlos, Fatima [4]
- 2 The names of a group of people attending a conference were recorded in a stack data structure. The first name stored in the stack was "Sofia".

|        |
|--------|
| ...    |
| Tariq  |
| Maya   |
| Jasper |
| Rina   |
| Rafael |
| Aisha  |
| Zara   |
| Sofia  |

Note that "Tariq" is currently in position 0 in the stack.

- a Compare and contrast the use of a binary search tree and a stack when searching for a specific item. [2]
  - b The tree is populated with the data from the stack. By considering only the data visible in the stack above, sketch the binary search tree that has been created from the items removed from the stack. [2]
- 3 Sketch a binary search tree that would allow the following output when traversed using an in-order traversal:  
Zebu, Tapir, Hedgehog, Falcon, Dugong, Bison, Armadillo [2]
  - 4 A hash table has been used to store a company's current stock. The hashing algorithm used is:  
stock number MOD 100.
    - a Determine the value returned by the hashing function when it is applied to stock number 1021. [1]
    - b Explain how a value is stored in a hash table. [1]
    - c Describe the steps involved in rehashing. [2]
  - 5 Given two sets,  $A = \{1, 2, 3, 4\}$  and  $B = \{3, 4, 5, 6\}$ , perform the following operations and provide the resulting set:
    - a Union [2]
    - b Intersection [2]
    - c Difference. [2]